

TITLE: OBJECTS AND CLASSES IN C++

Objectives:

- To understand the concept of classes and objects in C++.
- To use methods of classes to solve simple problems.
- To understand and implement static variables and static methods.

THEORY

In C++, class forms basis of object oriented programming. Before giving the name C++, it was called "C with classes." The sole purpose of class is to provide a tool for creating new data types called as abstract data type that can be used as conveniently as the built in types. A class is actually a user defined data type like structure in C.

Classes in C++

A class declaration is syntactically similar to a structure. Classes contain data as well as functions. The class can be termed as a collection of data members along with member functions. This binding of data and function together in a single unit is called encapsulation. The general format of declaring class is as follows:

```
class class_name {  
    private:  
        data-type data1;  
        //.....  
    public:  
        return-type function-name (data-type arg-list);  
        //.....  
};
```

The class definition starts with keyword class followed by class name. The body of a class is enclosed within curly braces and is terminated by a semicolon. The body consists of declarations called as members. These members are normally grouped into sections private, public and protected, called as access specifiers or visibility levels. The member declared in the private sections are hidden from other part of the program and are visible by member

functions within the class. The public access specifiers allow functions or data members in public section to be accessible to other part of the program too. When access specifiers are not specified, members will be private by default.

The access specifiers (private, public, protected) can occur more than once in the class definitions. That means we have private or public sections more than once. For example, the class definition can be done as:

```
class class_name{  
    private:  
        //private data and function  
    public:  
        //public data and members  
    private:  
        //more private data and function  
    public:  
        //more public data and members  
};
```

Objects and member access

The declaration of the class does not create any objects. It is just the specification of what a class contains. Sometimes a class is also called a specifier. After the class has been declared, we can create variables of its types called objects or instance of class. The syntax of creating object of any class is as follows:

```
class-name object_name;
```

where class-name refers to the name of the user defined class data type and the object-name refers to the name of the user defined objects. More than one objects can be declared simultaneously.

For example:

```
item menu i1, i2;
```


The instantiation of an object is similar to the declaration of variable of basic datatype. It is the definition of an object that actually creates objects in the program by setting memory space for its storage. The necessary memory space is allocated only when instance of class the object is created.

When an object of a class is created, its public members can be accessed by making use of the dot "." operator. The dot(.) operator is also called the member access operator. The private members cannot be accessed outside of the class using dot operator. For private members, we have to use getter functions. The syntax for accessing data members is:

object-name.data.member;

Similarly, the syntax for accessing member function is as follows:

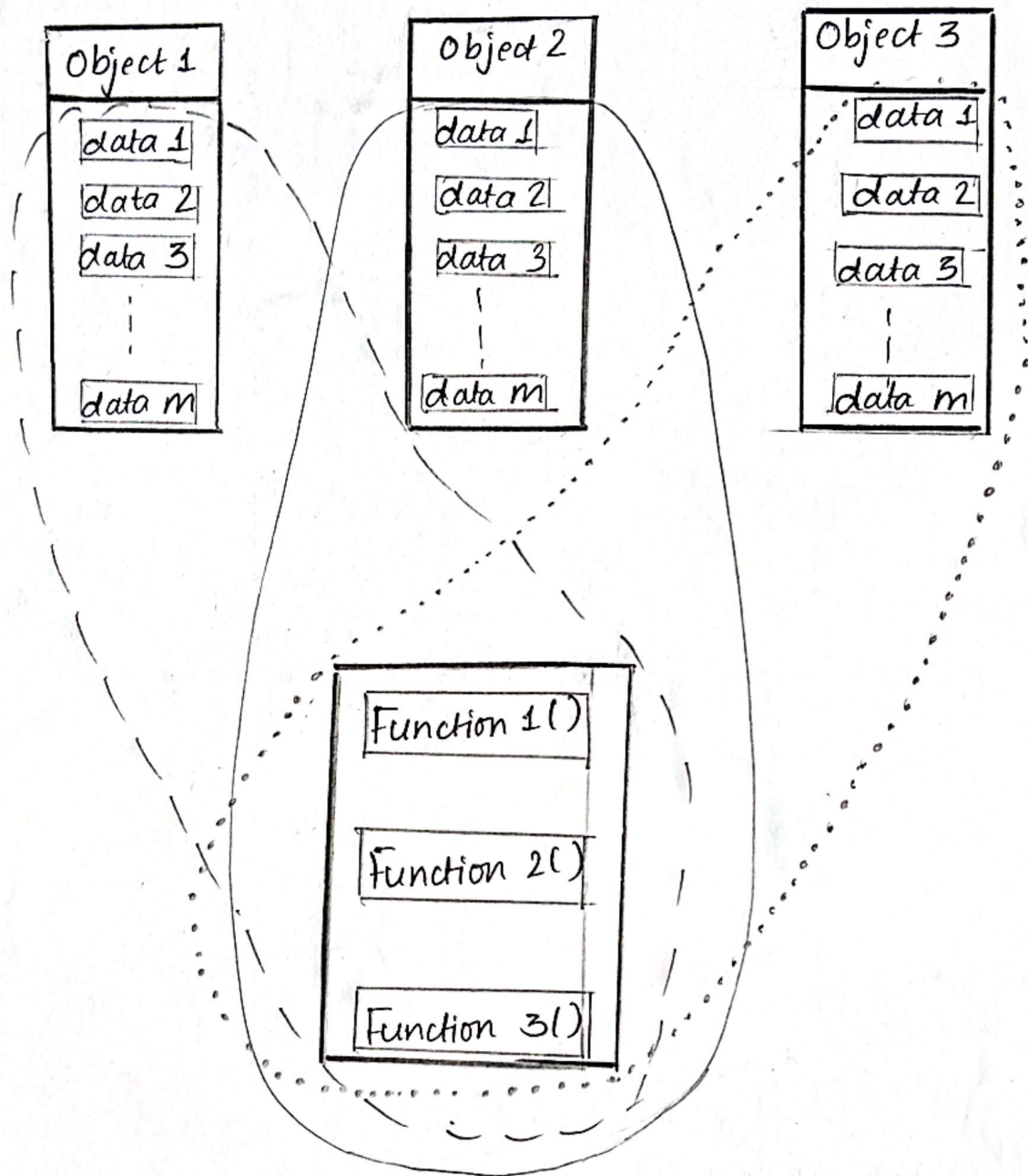
object_name.function_name(arguments);

Relation of Object, class and Memory

After the declaration of a class, memory isn't allocated for the data member of the class. The declaration of the class develops a template but data members cannot be manipulated unless an object is created. Then, when an object is created, memory is allocated only to its data members but not to member functions.

Member functions are created and stored in memory only when a class specification is declared. All ^{objects} of that class have access to the same area in the memory where the member functions are stored. It is logically true as the member functions are same for all objects and there is no point in allocating a separate copy for each and every object created using the same for class specification. But separate storage is allocated for every object's data member since they contain different values and allow them to handle their individual data.

The following figure illustrates the concept:



It is observed that 'n' objects of same class are created and data members of those objects are stored in distinct memory locations, whereas members of object 1 to object n are stored in the same memory area. Therefore, each object has a separate copy of data members and the different objects share the member functions among them.